

PRACTICAL-8

AIM: To implement Graph Searching using Depth First Search (DFS) and Breadth First Search (BFS) and understand the traversal of vertices in a graph.

Depth First Search (DFS):

ALGORITHM:

DFS Algorithm :-
input :- number of vertices & adjacency matrix
vertex start
output :- DFS traversal of the graph
step 1 :- start
step 2 :- read no. of vertices & adjacency matrix adj
step 3 :- mark the current vertex as visited
visited (nodes) = 1
step 4 :- start DFS from the given starting vertex
step 5 :- initialize all vertices as unvisited
step 6 :- display the current vertex
step 7 :- check all vertices from 0 to n-1
step 8 :- if a vertex is adjacent to current vertex & is unvisited
if (adj (nodes) [i] = 1 & visited [i] = 0)
call DFS recursively for that vertex
step 9 :- continue until all reachable vertices are visited
step 10 :- stop

PROGRAM/CODE:

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> graph[100];
bool visited[100];

void DFS(int vertex)
{
    Raja kiran (9240011865)
```

```
visited[vertex] = true;
cout << vertex << " ";

for (int i = 0; i < graph[vertex].size(); i++)
{
    int next = graph[vertex][i];

    if (!visited[next])
    {
        DFS(next);
    }
}

int main()
{
    int vertices, edges;

    cout << "Enter number of vertices: ";
    cin >> vertices;

    cout << "Enter number of edges: ";
    cin >> edges;

    cout << "Enter edges:\n";

    for (int i = 0; i < edges; i++)
    {
        int u, v;
        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    cout << "DFS Traversal: ";
    DFS(start);

    return 0;
}
```

OUTPUT:

```
Problems Output Debug Console Terminal Ports
nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/DFS
Enter number of vertices: 5
Enter number of edges: 5
Enter edges:
0 1
0 2
1 3
1 4
2 4
Enter starting vertex: 0
DFS Traversal: 0 1 3 4 2
nandhagopal@NGKs-MacBook-Air c++ %
```

TIME COMPLEXITY:

Time complexity :-

```

visited [vertex] = true; → 1
cout << vertex; → 1
for (int i = 0; i < graph[vertices].size(); i++) → E
{
    int next = graph[vertices][i]; → E
    if (!visited[next]) → E
    {
        DFS(next); → V
    }
}

```

therefore

$$T(V, E) = 1 + E + E + E + V$$

$$= 2 + 3E + V$$

taking the highest order term

$$T(V, E) = O(E + V)$$

1. BEST CASE:

Best case:
 If graph has very few nodes starting from the starting vertex
 → DFS visits the starting vertex & checking its adjacent vertex
 & visiting a vertex → $O(1)$
 & checking all vertices & edge → $O(V+E)$
 So,
 $T(V,E) = O(V+E)$

2. AVERAGE CASE:

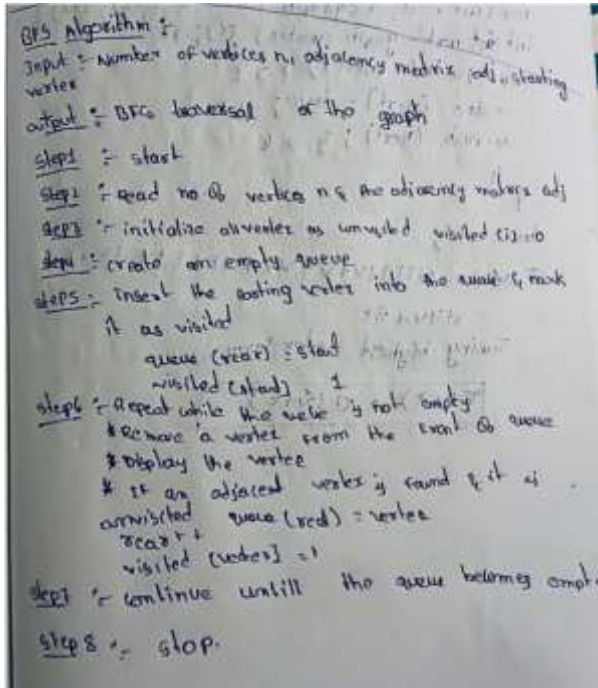
Average case:
 For a normally connected graph DFS visits most vertices & checks their edges
 * no. of vertices → V
 & no. of edge → E
 So,
 $T(V,E) = O(V+E)$

3. WORST CASE:

Worst case:
 All vertices are connected / reachable from the starting vertex
 → DFS visits every vertex & checks every edge
 So,
 $T(V,E) = O(V+E)$

Breadth First Search (BFS):

ALGORITHM:



PROGRAM/CODE:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

vector<int> graph[100];
bool visited[100];

void BFS(int start)
{
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty())
    {
        int vertex = q.front();
```

```
q.pop();

cout << vertex << " ";

for (int i = 0; i < graph[vertex].size(); i++)
{
    int next = graph[vertex][i];

    if (!visited[next])
    {
        visited[next] = true;
        q.push(next);
    }
}
}

int main()
{
    int vertices, edges;

    cout << "Enter number of vertices: ";
    cin >> vertices;

    cout << "Enter number of edges: ";
    cin >> edges;

    cout << "Enter edges:\n";

    for (int i = 0; i < edges; i++)
    {
        int u, v;
        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    cout << "BFS Traversal: ";
    BFS(start);

    return 0;
}
```

Raja kiran (92400118654)

}

OUTPUT:

```
nandhagopal@NGKs-MacBook-Air c++ % /Users/nandhagopal/Documents/CODING/c++/BFS
Enter number of vertices: 5
Enter number of edges: 5
Enter edges:
0 1
0 2
1 3
1 4
2 4
Enter starting vertex: 0
BFS Traversal: 0 1 2 3 4
nandhagopal@NGKs-MacBook-Air c++ %
```

TIME COMPLEXITY:

Time complexity

```

queue <int + > v; → 1
visited [start] = true; → 1
a. push (start); → 1
while (!q.empty ()) { → v
    v. front (); → v
    v. pop (); → v
    count << vertex; → v
    for (int i = 0; i < graph [vertex].size(); i++) { → E
        int next = graph [vertex] [i]; → E
        if (!visited [next]) { → E
            visited [next] = true; → E
            v. push (next); → E
        }
    }
}

```

therefore;

$$T(V, E) = 1 + 1 + 1 + v + v + v + v + E + E + E + E + E + E$$

$$= 3 + 4v + 5E$$

Taking highest order term

$$T(V, E) = O(V + E)$$

1. BEST CASE:

Best case:-
 BFS starts from the given vertex & checks its adjacent vertex
 #visiting vertex $\rightarrow O(1)$
 #checking vertices & edges $\rightarrow O(V+E)$
 So $T(V,E) = O(V+E)$

2. AVERAGE CASE:

Average case:-
 BFS visits vertices level by level & checks their adjacent edges
 #visiting $\rightarrow V$
 #edge $\rightarrow E$
 $T(V,E) = O(V+E)$

3. WORST CASE:

Worst case:-
 All vertices are connected & BFS visits the complete graph every vertex & edges & processed
 So $T(V,E) = O(V+E)$

CONCLUSION:

Thus, **Graph Searching using DFS and BFS** was successfully implemented, and the graph vertices were traversed correctly using both searching techniques.